

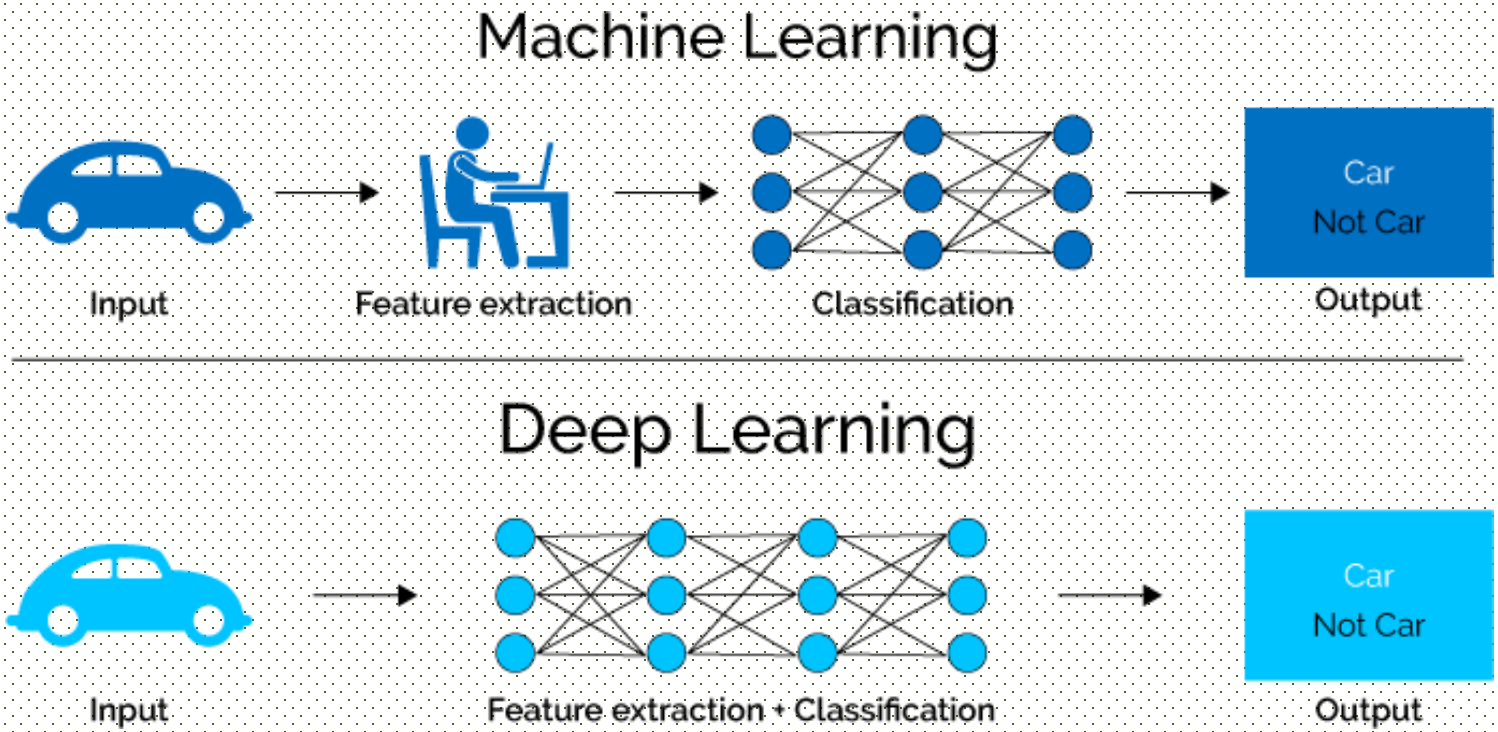


Redes Neuronales (II)

Ing. Jaime Antonio Huaytalla Pariona
jhuaytalla@uncp.edu.pe

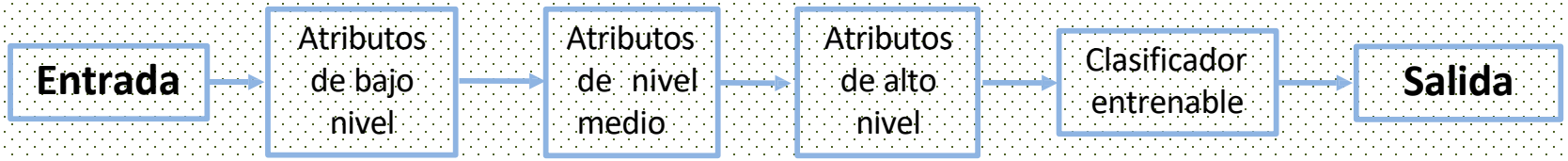
Introducción

- ¿Qué es Deep Learning?
  - Red neuronal con varias capas ocultas
  - Las capas ocultas realizan “identificación de características”
  - Solo funciona bien si hay muchos datos



Introducción

- En Deep Learning:

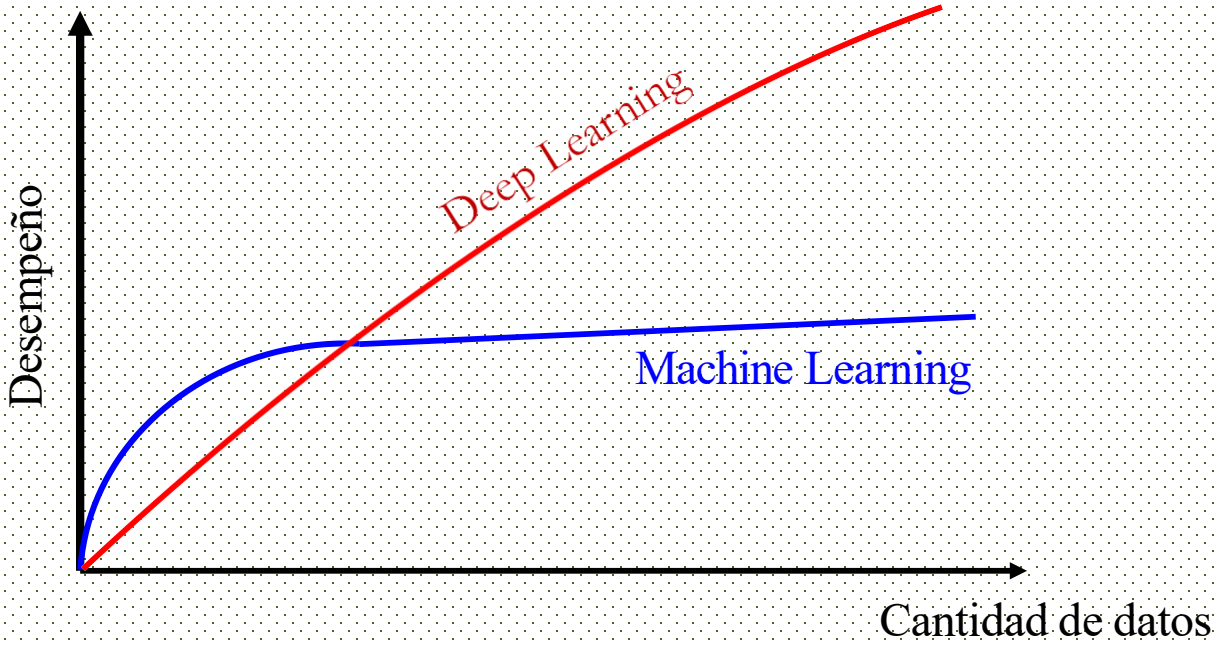


- Cada parte de una red neuronal profunda transforma su representación en un nivel de abstracción más alto

- Ejemplo
  - Imagen
    - Pixel → Borde → Textura → Parte → Objecto
  - Texto
    - Caracter → Palabra → Grupo de palabras → Proposición → Oración → Historia

Introducción

- Para poder aprender (y extraer características adecuadas), Deep Learning usa una gran cantidad de datos



- Gran cantidad de datos implica gran cantidad de parámetros
  - Se requiere mejores algoritmos de optimización
  - Es necesario realizar regularización

Regularización

Regularización de Norma

- Se utiliza alguna norma para regularizar los pesos
  - Usualmente: norma de Frobenius:

$$\left\|W^{[l]}\right\|_F^2 = \sum_{i=1}^{n^{[l-1]}} \sum_{j=1}^{n^{[l]}} (w_{ij}^{[l]})^2 \qquad W^{[l]} \in \mathbb{R}^{n^{[l]} \times n^{[l-1]}}$$

- Función de costo regularizada (asumiendo L capas y n instancias)

$$J = \frac{1}{n} \sum_{i=1}^{n} \ell(\hat{y}^{(i)}, y^{(i)}) + \frac{\lambda}{2n} \sum_{l=1}^{L} \left\|W^{[l]}\right\|_F^2$$

- Derivada de la función de regularización

$$\frac{\partial}{\partial w_{ij}^{[l]}} \left(\frac{\lambda}{2n} \sum_{i=1}^{n^{[l-1]}} \sum_{j=1}^{n^{[l]}} (w_{ij}^{[l]})^2 \right) = \frac{\lambda}{n} w_{ij}^{[l]} \quad \Rightarrow \quad \frac{\partial}{\partial W^{[l]}} \left(\frac{\lambda}{2n} \sum_{l=1}^{L} \left\|W^{[l]}\right\|_F^2 \right) = \frac{\lambda}{n} W^{[l]}$$

  - Esta derivada solo afecta la actualización de los pesos (no del bias)

Regularización

Regularización de Norma

- Actualización de parámetros

$$W^{[l]} = W^{[l]} - \alpha \left(dW^{[l]} + \frac{\lambda}{2n} W^{[l]} \right)$$

$$b^{[l]} = b^{[l]} - \alpha db^{[l]}$$

- La actualización de los pesos se puede factorizar: “decaimiento ponderado”

$$W^{[l]} = W^{[l]} - \alpha \frac{\lambda}{2n} W^{[l]} - \alpha dW^{[l]}$$

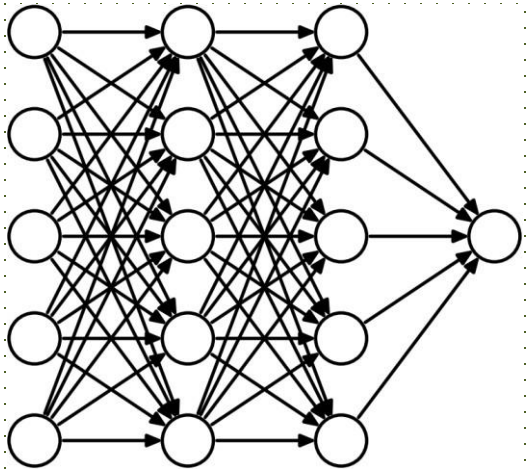
$$W^{[l]} = \underbrace{\left(1 - \alpha \frac{\lambda}{2n} \right)}_{\text{menor a 1}} W^{[l]} - \alpha dW^{[l]}$$

Regularización

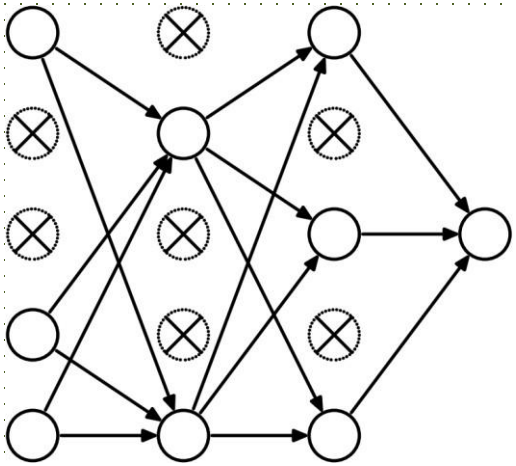
Dropout

- Retira un nodo de cada capa de manera aleatoria
  - Durante entrenamiento: cada neurona tiene probabilidad p de estar presente
  - Durante prueba: todas las neuronas están presentes

- Ejemplo: durante el entrenamiento



Red neuronal inicial

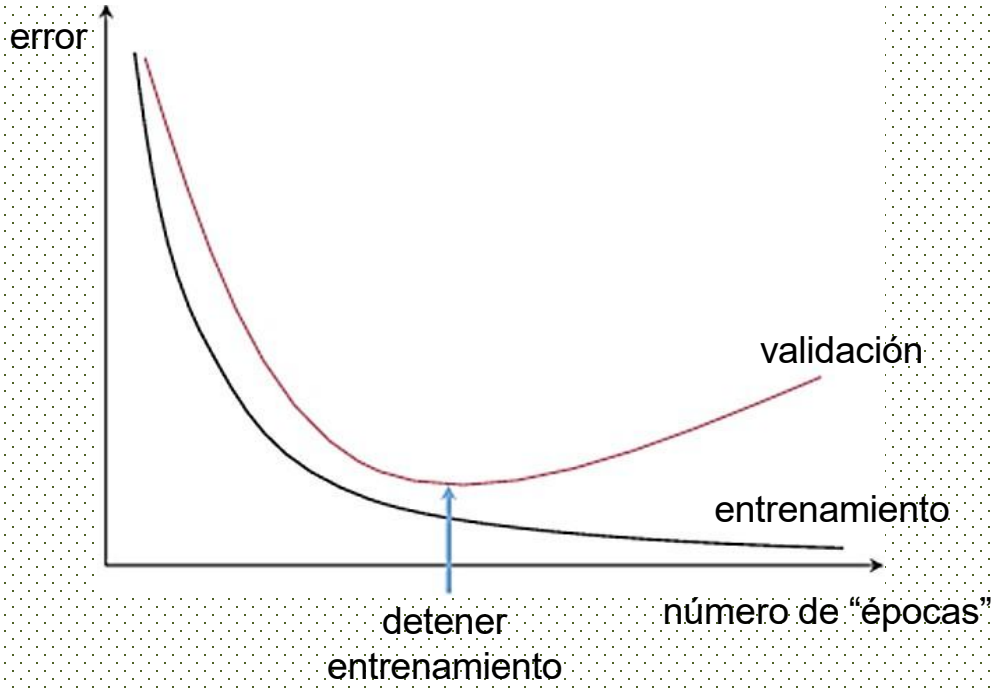


Red neuronal con “dropout”
(las X indican que no se usan)

Regularización

"Early Stopping"

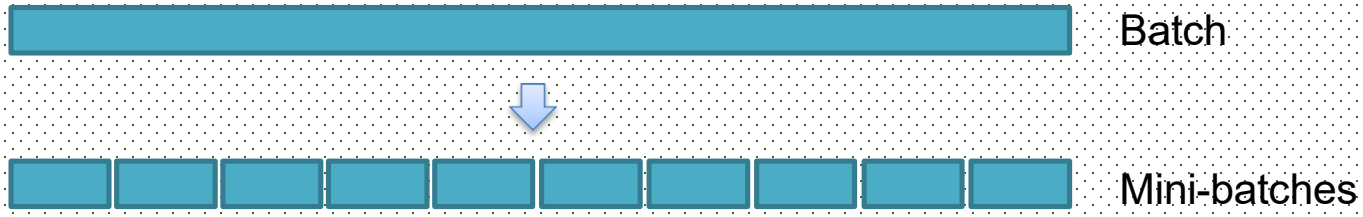
- Se analiza el error en el conjunto de entrenamiento y en conjunto de validación
- Se detiene el algoritmo antes de que realice "overfitting"



Algoritmos de Optimización

Descenso del Gradiente Mini-Batch

- Se divide todos los datos de entrenamiento (“batch”) en partes (“mini-batches”)



- El bucle utiliza cada uno de los mini-batches
  - En cada actualización de parámetros se utiliza 1 mini-batch (diferente)
  - La derivada que se usa es “aproximada” (no usa todo el batch)
  - Reduce el tiempo de entrenamiento

- Época (“epoch”): una “pasada” completa a todos los datos (todos los mini-batches)

Algoritmos de Optimización

Métodos Principales

- Son modificaciones del descenso del gradiente
- Métodos:
  - Descenso de gradiente con momentum
  - Gradiente acelerado de Nesterov
  - Adagrad
  - RMSprop
  - Algoritmo de Adam
  - AdaMax
  - Nadam
  - AMSGrad
  - AdamW
  - QHAdam
  - AggMo
  - YellowFin
  - QMH